

Chapter 3. Kafka Producers: Writing Messages to Kafka

Whether you use Kafka as a queue, message bus, or data storage platform, you will always use Kafka by creating a producer that writes data to Kafka, a consumer that reads data from Kafka, or an application that serves both roles.

For example, in a credit card transaction processing system, there will be a client application, perhaps an online store, responsible for sending each transaction to Kafka immediately when a payment is made. Another application is responsible for immediately checking this transaction against a rules engine and determining whether the transaction is approved or denied. The approve/deny response can then be written back to Kafka, and the response can propagate back to the online store where the transaction was initiated. A third application can read both transactions and the approval status from Kafka and store them in a database where analysts can later review the decisions and perhaps improve the rules engine.

Apache Kafka ships with built-in client APIs that developers can use when developing applications that interact with Kafka.

In this chapter we will learn how to use the Kafka producer, starting with an overview of its design and components. We will show how to create `KafkaProducer` and `ProducerRecord` objects, how to send records to Kafka, and how to handle the errors that Kafka may return. We'll then review the most important configuration options used to control the producer behavior. We'll conclude with a deeper look at how to use different partitioning methods and serializers, and how to write your own serializers and partitioners.

In [Chapter 4](#), we will look at Kafka's consumer client and reading data from Kafka.

THIRD-PARTY CLIENTS

In addition to the built-in clients, Kafka has a binary wire protocol. This means that it is possible for applications to read messages from Kafka or write messages to Kafka simply by sending the correct byte sequences to Kafka's network port. There are multiple clients that implement Kafka's wire protocol in different programming languages, giving simple ways to use Kafka not just in Java applications but also in languages like C++, Python, Go, and many more. Those clients are not part of the Apache Kafka project, but a list of non-Java clients is maintained in the [project wiki](#). The wire protocol and the external clients are outside the scope of the chapter.

Producer Overview

There are many reasons an application might need to write messages to Kafka: recording user activities for auditing or analysis, recording metrics, storing log messages, recording information from smart appliances,

communicating asynchronously with other applications, buffering information before writing to a database, and much more.

Those diverse use cases also imply diverse requirements: is every message critical, or can we tolerate loss of messages? Are we OK with accidentally duplicating messages? Are there any strict latency or throughput requirements we need to support?

In the credit card transaction processing example we introduced earlier, we can see that it is critical to never lose a single message or duplicate any messages. Latency should be low, but latencies up to 500 ms can be tolerated, and throughput should be very high—we expect to process up to a million messages a second.

A different use case might be to store click information from a website. In that case, some message loss or a few duplicates can be tolerated; latency can be high as long as there is no impact on the user experience. In other words, we don't mind if it takes a few seconds for the message to arrive at Kafka, as long as the next page loads immediately after the user clicks on a link. Throughput will depend on the level of activity we anticipate on our website.

The different requirements will influence the way you use the producer API to write messages to Kafka and the configuration you use.

While the producer API is very simple, there is a bit more that goes on under the hood of the producer when we send data. [Figure 3-1](#) shows the main steps involved in sending data to Kafka.

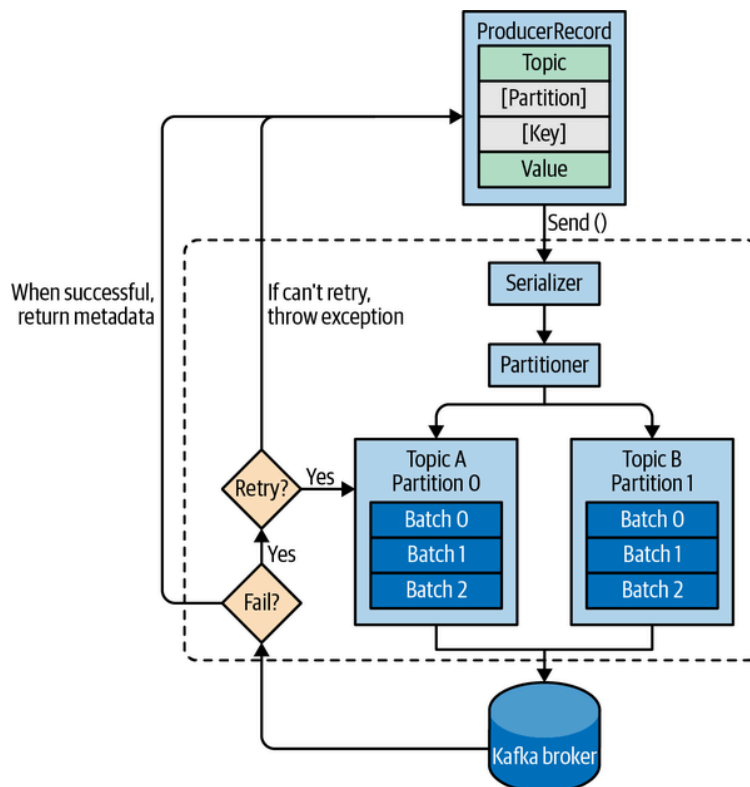


Figure 3-1. High-level overview of Kafka producer components

We start producing messages to Kafka by creating a `ProducerRecord`, which must include the topic we want to send the record to and a value. Optionally, we can also specify a key, a partition, a timestamp, and/or a collection of headers. Once we send the `ProducerRecord`, the first thing the producer will do is serialize the key and value objects to byte arrays so they can be sent over the network.

Next, if we didn't explicitly specify a partition, the data is sent to a partitioner. The partitioner will choose a partition for us, usually based on the `ProducerRecord` key. Once a partition is selected, the producer knows which topic and partition the record will go to. It then adds the record to a batch of records that will also be sent to the same topic and partition. A separate thread is responsible for sending those batches of records to the appropriate Kafka brokers.

When the broker receives the messages, it sends back a response. If the messages were successfully written to Kafka, it will return a `RecordMetadata` object with the topic, partition, and the offset of the record within the partition. If the broker failed to write the messages, it will return an error. When the producer receives an error, it may retry sending the message a few more times before giving up and returning an error.

Constructing a Kafka Producer

The first step in writing messages to Kafka is to create a producer object with the properties you want to pass to the producer. A Kafka producer has three mandatory properties:

`bootstrap.servers`

List of `host:port` pairs of brokers that the producer will use to establish initial connection to the Kafka cluster. This list doesn't need to include all brokers, since the producer will get more information after the initial connection. But it is recommended to include at least two, so in case one broker goes down, the producer will still be able to connect to the cluster.

`key.serializer`

Name of a class that will be used to serialize the keys of the records we will produce to Kafka. Kafka brokers expect byte arrays as keys and values of messages. However, the producer interface allows, using parameterized types, any Java object to be sent as a key and value. This makes for very readable code, but it also means that the producer has to know how to convert these objects to byte arrays.

`key.serializer` should be set to a name of a class that implements the

`org.apache.kafka.common.serialization.Serializer` interface. The producer will use this class to serialize the key object to a byte array. The Kafka client package includes

`ByteArraySerializer` (which doesn't do much), `StringSerializer`, `IntegerSerializer`, and much more, so if you use common types, there is no need to implement your own serializers. Setting `key.serializer` is required even if you intend to send only values, but you can use the `void` type for the key and the `VoidSerializer`.

`value.serializer`

Name of a class that will be used to serialize the values of the records we will produce to Kafka. The same way you set

`key.serializer` to a name of a class that will serialize the message key object to a byte array, you set `value.serializer` to a class that will serialize the message value object.

The following code snippet shows how to create a new producer by setting just the mandatory parameters and using defaults for everything

else:

```
Properties kafkaProps = new Properties(); ❶
kafkaProps.put("bootstrap.servers", "broker1:9092,broker2:9092");

kafkaProps.put("key.serializer",
    "org.apache.kafka.common.serialization.StringSerializer"); ❷
kafkaProps.put("value.serializer",
    "org.apache.kafka.common.serialization.StringSerializer");

producer = new KafkaProducer<String, String>(kafkaProps); ❸
```

- ❶ We start with a `Properties` object.
- ❷ Since we plan on using strings for message key and value, we use the built-in `StringSerializer`.
- ❸ Here we create a new producer by setting the appropriate key and value types and passing the `Properties` object.

With such a simple interface, it is clear that most of the control over producer behavior is done by setting the correct configuration properties. Apache Kafka documentation covers all the [configuration options](#), and we will go over the important ones later in this chapter.

Once we instantiate a producer, it is time to start sending messages. There are three primary methods of sending messages:

Fire-and-forget

We send a message to the server and don't really care if it arrives successfully or not. Most of the time, it will arrive successfully, since Kafka is highly available and the producer will retry sending messages automatically. However, in case of nonretriable errors or timeout, messages will get lost and the application will not get any information or exceptions about this.

Synchronous send

Technically, Kafka producer is always asynchronous—we send a message and the `send()` method returns a `Future` object. However, we use `get()` to wait on the `Future` and see if the `send()` was successful or not before sending the next record.

Asynchronous send

We call the `send()` method with a callback function, which gets triggered when it receives a response from the Kafka broker.

In the examples that follow, we will see how to send messages using these methods and how to handle the different types of errors that might occur.

While all the examples in this chapter are single threaded, a producer object can be used by multiple threads to send messages.

Sending a Message to Kafka

The simplest way to send a message is as follows:

```
ProducerRecord<String, String> record =
    new ProducerRecord<>("CustomerCountry", "Precision Products",
        "France"); ❶
```

```

try {
    producer.send(record); ❷
} catch (Exception e) {
    e.printStackTrace(); ❸
}

```

- ❶ The producer accepts `ProducerRecord` objects, so we start by creating one. `ProducerRecord` has multiple constructors, which we will discuss later. Here we use one that requires the name of the topic we are sending data to, which is always a string, and the key and value we are sending to Kafka, which in this case are also strings. The types of the key and value must match our `key serializer` and `value serializer` objects.
- ❷ We use the producer object `send()` method to send the `ProducerRecord`. As we've seen in the producer architecture diagram in [Figure 3-1](#), the message will be placed in a buffer and will be sent to the broker in a separate thread. The `send()` method returns a [Java Future object](#) with `RecordMetadata`, but since we simply ignore the returned value, we have no way of knowing whether the message was sent successfully or not. This method of sending messages can be used when dropping a message silently is acceptable. This is not typically the case in production applications.
- ❸ While we ignore errors that may occur while sending messages to Kafka brokers or in the brokers themselves, we may still get an exception if the producer encountered errors before sending the message to Kafka. Those can be, for example, a `SerializationException` when it fails to serialize the message, a `BufferExhaustedException` or `TimeoutException` if the buffer is full, or an `InterruptedException` if the sending thread was interrupted.

Sending a Message Synchronously

Sending a message synchronously is simple but still allows the producer to catch exceptions when Kafka responds to the produce request with an error, or when send retries were exhausted. The main trade-off involved is performance. Depending on how busy the Kafka cluster is, brokers can take anywhere from 2 ms to a few seconds to respond to produce requests. If you send messages synchronously, the sending thread will spend this time waiting and doing nothing else, not even sending additional messages. This leads to very poor performance, and as a result, synchronous sends are usually not used in production applications (but are very common in code examples).

The simplest way to send a message synchronously is as follows:

```

ProducerRecord<String, String> record =
    new ProducerRecord<>("CustomerCountry", "Precision Products", "France");
try {
    producer.send(record).get(); ❶
} catch (Exception e) {
    e.printStackTrace(); ❷
}

```

- ❶ Here, we are using `Future.get()` to wait for a reply from Kafka. This method will throw an exception if the record is not sent successfully to Kafka. If there were no errors, we will get a

`RecordMetadata` object that we can use to retrieve the offset the message was written to and other metadata.

- ❷ If there were any errors before or while sending the record to Kafka, we will encounter an exception. In this case, we just print any exception we ran into.

`KafkaProducer` has two types of errors. *Retriable* errors are those that can be resolved by sending the message again. For example, a connection error can be resolved because the connection may get reestablished. A “not leader for partition” error can be resolved when a new leader is elected for the partition and the client metadata is refreshed.

`KafkaProducer` can be configured to retry those errors automatically, so the application code will get retrievable exceptions only when the number of retries was exhausted and the error was not resolved. Some errors will not be resolved by retrying—for example, “Message size too large.” In those cases, `KafkaProducer` will not attempt a retry and will return the exception immediately.

Sending a Message Asynchronously

Suppose the network round-trip time between our application and the Kafka cluster is 10 ms. If we wait for a reply after sending each message, sending 100 messages will take around 1 second. On the other hand, if we just send all our messages and not wait for any replies, then sending 100 messages will barely take any time at all. In most cases, we really don’t need a reply—Kafka sends back the topic, partition, and offset of the record after it was written, which is usually not required by the sending app. On the other hand, we do need to know when we failed to send a message completely so we can throw an exception, log an error, or perhaps write the message to an “errors” file for later analysis.

To send messages asynchronously and still handle error scenarios, the producer supports adding a callback when sending a record. Here is an example of how we use a callback:

```
private class DemoProducerCallback implements Callback { ❶
    @Override
    public void onCompletion(RecordMetadata recordMetadata, Exception e) {
        if (e != null) {
            e.printStackTrace(); ❷
        }
    }
}

ProducerRecord<String, String> record =
    new ProducerRecord<>("CustomerCountry", "Biomedical Materials", "USA"); ❸
producer.send(record, new DemoProducerCallback()); ❹
```

- ❶ To use callbacks, you need a class that implements the `org.apache.kafka.clients.producer.Callback` interface, which has a single function—`onCompletion()`.
- ❷ If Kafka returned an error, `onCompletion()` will have a nonnull exception. Here we “handle” it by printing, but production code will probably have more robust error handling functions.
- ❸ The records are the same as before.
- ❹ And we pass a `Callback` object along when sending the record.

WARNING

The callbacks execute in the producer's main thread. This guarantees that when we send two messages to the same partition one after another, their callbacks will be executed in the same order that we sent them. But it also means that the callback should be reasonably fast to avoid delaying the producer and preventing other messages from being sent. It is not recommended to perform a blocking operation within the callback. Instead, you should use another thread to perform any blocking operation concurrently.

Configuring Producers

So far we've seen very few configuration parameters for the producers—just the mandatory `bootstrap.servers` URI and serializers.

The producer has a large number of configuration parameters that are documented in [Apache Kafka documentation](#), and many have reasonable defaults, so there is no reason to tinker with every single parameter. However, some of the parameters have a significant impact on memory use, performance, and reliability of the producers. We will review those here.

`client.id`

`client.id` is a logical identifier for the client and the application it is used in. This can be any string and will be used by the brokers to identify messages sent from the client. It is used in logging and metrics and for quotas. Choosing a good client name will make troubleshooting much easier—it is the difference between “We are seeing a high rate of authentication failures from IP 104.27.155.134” and “Looks like the Order Validation service is failing to authenticate—can you ask Laura to take a look?”

`acks`

The `acks` parameter controls how many partition replicas must receive the record before the producer can consider the write successful. By default, Kafka will respond that the record was written successfully after the leader received the record (release 3.0 of Apache Kafka is expected to change this default). This option has a significant impact on the durability of written messages, and depending on your use case, the default may not be the best choice. [Chapter 7](#) discusses Kafka's reliability guarantees in depth, but for now let's review the three allowed values for the `acks` parameter:

`acks=0`

The producer will not wait for a reply from the broker before assuming the message was sent successfully. This means that if something goes wrong and the broker does not receive the message, the producer will not know about it, and the message will be lost. However, because the producer is not waiting for any response from the server, it can send messages as fast as the network will support, so this setting can be used to achieve very high throughput.

`acks=1`

The producer will receive a success response from the broker the moment the leader replica receives the message. If the message can't be written to the leader (e.g., if the leader crashed and a new

leader was not elected yet), the producer will receive an error response and can retry sending the message, avoiding potential loss of data. The message can still get lost if the leader crashes and the latest messages were not yet replicated to the new leader.

`acks=all`

The producer will receive a success response from the broker once all in sync replicas receive the message. This is the safest mode since you can make sure more than one broker has the message and that the message will survive even in case of a crash (more information on this in [Chapter 6](#)). However, the latency we discussed in the `acks=1` case will be even higher, since we will be waiting for more than just one broker to receive the message.

TIP

You will see that with lower and less reliable `acks` configuration, the producer will be able to send records faster. This means that you trade off reliability for *producer latency*. However, *end-to-end latency* is measured from the time a record was produced until it is available for consumers to read and is identical for all three options. The reason is that, in order to maintain consistency, Kafka will not allow consumers to read records until they are written to all in sync replicas. Therefore, if you care about end-to-end latency, rather than just the producer latency, there is no trade-off to make: you will get the same end-to-end latency if you choose the most reliable option.

Message Delivery Time

The producer has multiple configuration parameters that interact to control one of the behaviors that are of most interest to developers: how long will it take until a call to `send()` will succeed or fail. This is the time we are willing to spend until Kafka responds successfully, or until we are willing to give up and admit defeat.

The configurations and their behaviors were modified several times over the years. We will describe here the latest implementation, introduced in Apache Kafka 2.1.

Since Apache Kafka 2.1, we divide the time spent sending a `ProduceRecord` into two time intervals that are handled separately:

- Time until an async call to `send()` returns. During this interval, the thread that called `send()` will be blocked.
- From the time an async call to `send()` returned successfully until the callback is triggered (with success or failure). This is the same as from the point a `ProduceRecord` was placed in a batch for sending until Kafka responds with success, nonretriable failure, or we run out of time allocated for sending.

NOTE

If you use `send()` synchronously, the sending thread will block for both time intervals continuously, and you won't be able to tell how much time was spent in each. We'll discuss the common and recommended case, where `send()` is used asynchronously, with a callback.

The flow of data within the producer and how the different configuration parameters affect each other can be summarized in [Figure 3-2](#).¹

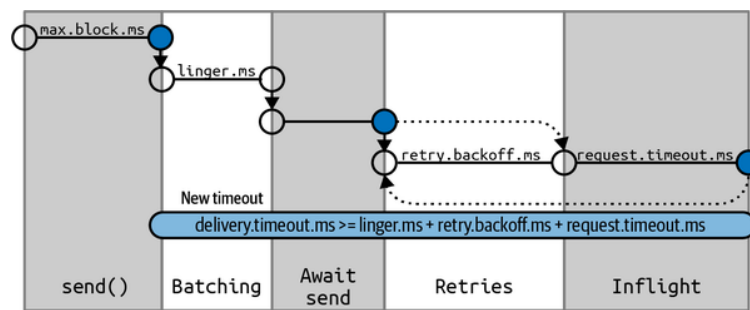


Figure 3-2. Sequence diagram of delivery time breakdown inside Kafka producer

We'll go through the different configuration parameters used to control the time spent waiting in these two intervals and how they interact.

max.block.ms

This parameter controls how long the producer may block when calling `send()` and when explicitly requesting metadata via `partitionsFor()`. Those methods may block when the producer's send buffer is full or when metadata is not available. When `max.block.ms` is reached, a timeout exception is thrown.

delivery.timeout.ms

This configuration will limit the amount of time spent from the point a record is ready for sending (`send()` returned successfully and the record is placed in a batch) until either the broker responds or the client gives up, including time spent on retries. As you can see in [Figure 3-2](#), this time should be greater than `linger.ms` and `request.timeout.ms`. If you try to create a producer with an inconsistent timeout configuration, you will get an exception. Messages can be successfully sent much faster than `delivery.timeout.ms`, and typically will.

If the producer exceeds `delivery.timeout.ms` while retrying, the callback will be called with the exception that corresponds to the error that the broker returned before retrying. If `delivery.timeout.ms` is exceeded while the record batch was still waiting to be sent, the callback will be called with a timeout exception.

TIP

You can configure the delivery timeout to the maximum time you'll want to wait for a message to be sent, typically a few minutes, and then leave the default number of retries (virtually infinite). With this configuration, the producer will keep retrying for as long as it has time to keep trying (or until it succeeds). This is a much more reasonable way to think about retries. Our normal process for tuning retries is: "In case of a broker crash, it typically takes leader election 30 seconds to complete, so let's keep retrying for 120 seconds just to be on the safe side." Instead of converting this mental dialog to number of retries and time between retries, you just configure `delivery.timeout.ms` to 120.

request.timeout.ms

This parameter controls how long the producer will wait for a reply from the server when sending data. Note that this is the time spent waiting on each producer request before giving up; it does not include retries, time spent before sending, and so on. If the timeout is reached without reply, the producer will either retry sending or complete the callback with a `TimeoutException`.

retries and retry.backoff.ms

When the producer receives an error message from the server, the error could be transient (e.g., a lack of leader for a partition). In this case, the value of the `retries` parameter will control how many times the producer will retry sending the message before giving up and notifying the client of an issue. By default, the producer will wait 100 ms between retries, but you can control this using the `retry.backoff.ms` parameter.

We recommend against using these parameters in the current version of Kafka. Instead, test how long it takes to recover from a crashed broker (i.e., how long until all partitions get new leaders), and set `delivery.timeout.ms` such that the total amount of time spent retrying will be longer than the time it takes the Kafka cluster to recover from the crash—otherwise, the producer will give up too soon.

Not all errors will be retried by the producer. Some errors are not transient and will not cause retries (e.g., “message too large” error). In general, because the producer handles retries for you, there is no point in handling retries within your own application logic. You will want to focus your efforts on handling nonretriable errors or cases where retry attempts were exhausted.

TIP

If you want to completely disable retries, setting `retries=0` is the only way to do so.

linger.ms

`linger.ms` controls the amount of time to wait for additional messages before sending the current batch. `KafkaProducer` sends a batch of messages either when the current batch is full or when the `linger.ms` limit is reached. By default, the producer will send messages as soon as there is a sender thread available to send them, even if there’s just one message in the batch. By setting `linger.ms` higher than 0, we instruct the producer to wait a few milliseconds to add additional messages to the batch before sending it to the brokers. This increases latency a little and significantly increases throughput—the overhead per message is much lower, and compression, if enabled, is much better.

buffer.memory

This config sets the amount of memory the producer will use to buffer messages waiting to be sent to brokers. If messages are sent by the application faster than they can be delivered to the server, the producer may run out of space, and additional `send()` calls will block for `max.block.ms` and wait for space to free up before throwing an exception. Note that unlike most producer exceptions, this timeout is thrown by `send()` and not by the resulting `Future`.

compression.type

By default, messages are sent uncompressed. This parameter can be set to `snappy`, `gzip`, `lz4`, or `zstd`, in which case the corresponding compression algorithms will be used to compress the data before sending it to the brokers. Snappy compression was invented by Google to provide decent compression ratios with low CPU overhead and good performance,

so it is recommended in cases where both performance and bandwidth are a concern. Gzip compression will typically use more CPU and time but results in better compression ratios, so it is recommended in cases where network bandwidth is more restricted. By enabling compression, you reduce network utilization and storage, which is often a bottleneck when sending messages to Kafka.

batch.size

When multiple records are sent to the same partition, the producer will batch them together. This parameter controls the amount of memory in bytes (not messages!) that will be used for each batch. When the batch is full, all the messages in the batch will be sent. However, this does not mean that the producer will wait for the batch to become full. The producer will send half-full batches and even batches with just a single message in them. Therefore, setting the batch size too large will not cause delays in sending messages; it will just use more memory for the batches. Setting the batch size too small will add some overhead because the producer will need to send messages more frequently.

max.in.flight.requests.per.connection

This controls how many message batches the producer will send to the server without receiving responses. Higher settings can increase memory usage while improving throughput. [Apache's wiki experiments show](#) that in a single-DC environment, the throughput is maximized with only 2 in-flight requests; however, the default value is 5 and shows similar performance.

ORDERING GUARANTEES

Apache Kafka preserves the order of messages within a partition. This means that if messages are sent from the producer in a specific order, the broker will write them to a partition in that order and all consumers will read them in that order. For some use cases, order is very important. There is a big difference between depositing \$100 in an account and later withdrawing it, and the other way around! However, some use cases are less sensitive.

Setting the `retries` parameter to nonzero and the `max.in.flight.requests.per.connection` to more than 1 means that it is possible that the broker will fail to write the first batch of messages, succeed in writing the second (which was already in-flight), and then retry the first batch and succeed, thereby reversing the order.

Since we want at least two in-flight requests for performance reasons, and a high number of retries for reliability reasons, the best solution is to set `enable.idempotence=true`. This guarantees message ordering with up to five in-flight requests and also guarantees that retries will not introduce duplicates. [Chapter 8](#) discusses the idempotent producer in depth.

max.request.size

This setting controls the size of a produce request sent by the producer. It caps both the size of the largest message that can be sent and the number of messages that the producer can send in one request. For example, with a default maximum request size of 1 MB, the largest message you can send is 1 MB, or the producer can batch 1,024 messages of size 1 KB each into one request. In addition, the broker has its own limit on the size of the largest message it will accept (`message.max.bytes`). It is usually a

good idea to have these configurations match, so the producer will not attempt to send messages of a size that will be rejected by the broker.

receive.buffer.bytes and send.buffer.bytes

These are the sizes of the TCP send and receive buffers used by the sockets when writing and reading data. If these are set to `-1`, the OS defaults will be used. It is a good idea to increase these when producers or consumers communicate with brokers in a different datacenter, because those network links typically have higher latency and lower bandwidth.

enable.idempotence

Starting in version 0.11, Kafka supports *exactly once* semantics. Exactly once is a fairly large topic, and we'll dedicate an entire chapter to it, but idempotent producer is a simple and highly beneficial part of it.

Suppose you configure your producer to maximize reliability: `acks=all` and a decently large `delivery.timeout.ms` to allow sufficient retries. These make sure each message will be written to Kafka at least once. In some cases, this means that messages will be written to Kafka more than once. For example, imagine that a broker received a record from the producer, wrote it to local disk, and the record was successfully replicated to other brokers, but then the first broker crashed before sending a response to the producer. The producer will wait until it reaches `request.timeout.ms` and then retry. The retry will go to the new leader that already has a copy of this record since the previous write was replicated successfully. You now have a duplicate record.

To avoid this, you can set `enable.idempotence=true`. When the idempotent producer is enabled, the producer will attach a sequence number to each record it sends. If the broker receives records with the same sequence number, it will reject the second copy and the producer will receive the harmless `DuplicateSequenceException`.

NOTE

Enabling idempotence requires `max.in.flight.requests.per.connection` to be less than or equal to 5, `retries` to be greater than 0, and `acks=all`. If incompatible values are set, a `ConfigException` will be thrown.

Serializers

As seen in previous examples, producer configuration includes mandatory serializers. We've seen how to use the default `String` serializer. Kafka also includes serializers for integers, `ByteArrays`, and many more, but this does not cover most use cases. Eventually, you will want to be able to serialize more generic records.

We will start by showing how to write your own serializer and then introduce the Avro serializer as a recommended alternative.

Custom Serializers

When the object you need to send to Kafka is not a simple string or integer, you have a choice of either using a generic serialization library like Avro, Thrift, or Protobuf to create records, or creating a custom serialization for objects you are already using. We highly recommend using a

generic serialization library. In order to understand how the serializers work and why it is a good idea to use a serialization library, let's see what it takes to write your own custom serializer.

Suppose that instead of recording just the customer name, you create a simple class to represent customers:

```
public class Customer {
    private int customerID;
    private String customerName;

    public Customer(int ID, String name) {
        this.customerID = ID;
        this.customerName = name;
    }

    public int getID() {
        return customerID;
    }

    public String getName() {
        return customerName;
    }
}
```

Now suppose we want to create a custom serializer for this class. It will look something like this:

```
import org.apache.kafka.common.errors.SerializationException;

import java.nio.ByteBuffer;
import java.util.Map;

public class CustomerSerializer implements Serializer<Customer> {

    @Override
    public void configure(Map configs, boolean isKey) {
        // nothing to configure
    }

    @Override
    /**
     * We are serializing Customer as:
     * 4 byte int representing customerId
     * 4 byte int representing length of customerName in UTF-8 bytes (0 if
     *   name is Null)
     * N bytes representing customerName in UTF-8
     */
    public byte[] serialize(String topic, Customer data) {
        try {
            byte[] serializedName;
            int stringSize;
            if (data == null)
                return null;
            else {
                if (data.getName() != null) {
                    serializedName = data.getName().getBytes("UTF-8");
                    stringSize = serializedName.length;
                } else {
                    serializedName = new byte[0];
                    stringSize = 0;
                }
            }

            ByteBuffer buffer = ByteBuffer.allocate(4 + 4 + stringSize);
            buffer.putInt(data.getID());
```

```

        buffer.putInt(stringSize);
        buffer.put(serializedName);

        return buffer.array();
    } catch (Exception e) {
        throw new SerializationException(
            "Error when serializing Customer to byte[] " + e);
    }
}

@Override
public void close() {
    // nothing to close
}
}

```

Configuring a producer with this `CustomerSerializer` will allow you to define `ProducerRecord<String, Customer>`, and send `Customer` data and pass `Customer` objects directly to the producer. This example is pretty simple, but you can see how fragile the code is. If we ever have too many customers, for example, and need to change `customerID` to `Long`, or if we ever decide to add a `startDate` field to `Customer`, we will have a serious issue in maintaining compatibility between old and new messages. Debugging compatibility issues between different versions of serializers and deserializers is fairly challenging: you need to compare arrays of raw bytes. To make matters even worse, if multiple teams in the same company end up writing `Customer` data to Kafka, they will all need to use the same serializers and modify the code at the exact same time.

For these reasons, we recommend using existing serializers and deserializers such as JSON, Apache Avro, Thrift, or Protobuf. In the following section, we will describe Apache Avro and then show how to serialize Avro records and send them to Kafka.

Serializing Using Apache Avro

Apache Avro is a language-neutral data serialization format. The project was created by Doug Cutting to provide a way to share data files with a large audience.

Avro data is described in a language-independent schema. The schema is usually described in JSON, and the serialization is usually to binary files, although serializing to JSON is also supported. Avro assumes that the schema is present when reading and writing files, usually by embedding the schema in the files themselves.

One of the most interesting features of Avro, and what makes it a good fit for use in a messaging system like Kafka, is that when the application that is writing messages switches to a new but compatible schema, the applications reading the data can continue processing messages without requiring any change or update.

Suppose the original schema was:

```

{
  "namespace": "customerManagement.avro",
  "type": "record",
  "name": "Customer",
  "fields": [
    { "name": "id", "type": "int" },
    { "name": "name", "type": "string" },
    { "name": "faxNumber", "type": [ "null", "string" ], "default": "null" }
  ]
}

```

```
]
}
```

- ❶ `id` and `name` fields are mandatory, while `faxNumber` is optional and defaults to `null`.

We used this schema for a few months and generated a few terabytes of data in this format. Now suppose we decide that in the new version, we will upgrade to the 21st century and will no longer include a fax number field and will instead use an email field.

The new schema would be:

```
{ "namespace": "customerManagement.avro",
  "type": "record",
  "name": "Customer",
  "fields": [
    { "name": "id", "type": "int" },
    { "name": "name", "type": "string" },
    { "name": "email", "type": [ "null", "string" ], "default": "null" }
  ]
}
```

Now, after upgrading to the new version, old records will contain `faxNumber` and new records will contain `email`. In many organizations, upgrades are done slowly and over many months. So we need to consider how pre-upgrade applications that still use the fax numbers and post-upgrade applications that use email will be able to handle all the events in Kafka.

The reading application will contain calls to methods similar to `getName()`, `getId()`, and `getFaxNumber()`. If it encounters a message written with the new schema, `getName()` and `getId()` will continue working with no modification, but `getFaxNumber()` will return `null` because the message will not contain a fax number.

Now suppose we upgrade our reading application and it no longer has the `getFaxNumber()` method but rather `getEmail()`. If it encounters a message written with the old schema, `getEmail()` will return `null` because the older messages do not contain an email address.

This example illustrates the benefit of using Avro: even though we changed the schema in the messages without changing all the applications reading the data, there will be no exceptions or breaking errors and no need for expensive updates of existing data.

However, there are two caveats to this scenario:

- The schema used for writing the data and the schema expected by the reading application must be compatible. The Avro documentation includes [compatibility rules](#).
- The deserializer will need access to the schema that was used when writing the data, even when it is different from the schema expected by the application that accesses the data. In Avro files, the writing schema is included in the file itself, but there is a better way to handle this for Kafka messages. We will look at that next.

Using Avro Records with Kafka

Unlike Avro files, where storing the entire schema in the data file is associated with a fairly reasonable overhead, storing the entire schema in each record will usually more than double the record size. However, Avro still requires the entire schema to be present when reading the record, so we need to locate the schema elsewhere. To achieve this, we follow a common architecture pattern and use a *Schema Registry*. The Schema Registry is not part of Apache Kafka, but there are several open source options to choose from. We'll use the Confluent Schema Registry for this example. You can find the Schema Registry code on [GitHub](#), or you can install it as part of the [Confluent Platform](#). If you decide to use the Schema Registry, we recommend checking [the documentation on Confluent](#).

The idea is to store all the schemas used to write data to Kafka in the registry. Then we simply store the identifier for the schema in the record we produce to Kafka. The consumers can then use the identifier to pull the record out of the Schema Registry and deserialize the data. The key is that all this work—storing the schema in the registry and pulling it up when required—is done in the serializers and deserializers. The code that produces data to Kafka simply uses the Avro serializer just like it would any other serializer. [Figure 3-3](#) demonstrates this process.

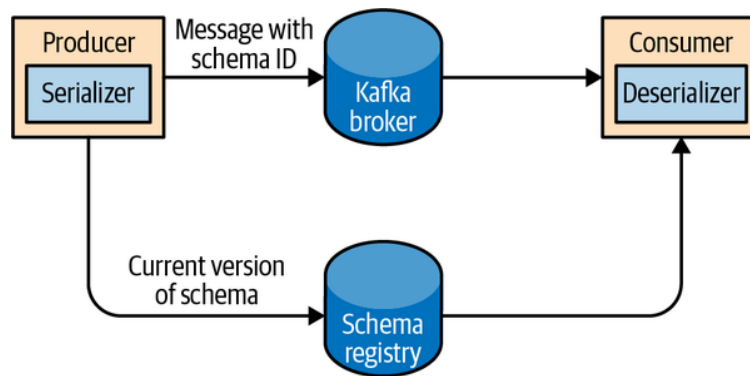


Figure 3-3. Flow diagram of serialization and deserialization of Avro records

Here is an example of how to produce generated Avro objects to Kafka (see the [Avro documentation](#) for how to generate objects from Avro schemas):

```
Properties props = new Properties();

props.put("bootstrap.servers", "localhost:9092");
props.put("key.serializer",
    "io.confluent.kafka.serializers.KafkaAvroSerializer");
props.put("value.serializer",
    "io.confluent.kafka.serializers.KafkaAvroSerializer"); ❶
props.put("schema.registry.url", schemaUrl); ❷

String topic = "customerContacts";

Producer<String, Customer> producer = new KafkaProducer<>(props); ❸

// We keep producing new events until someone ctrl-c
while (true) {
    Customer customer = CustomerGenerator.getNext(); ❹
    System.out.println("Generated customer " +
        customer.toString());
    ProducerRecord<String, Customer> record =
        new ProducerRecord<>(topic, customer.getName(), customer); ❺
    producer.send(record); ❻
}
```

- ❶ We use the `KafkaAvroSerializer` to serialize our objects with Avro. Note that the `KafkaAvroSerializer` can also handle primitives, which is why we can later use `String` as the record key and our `Customer` object as the value.
- ❷ `schema.registry.url` is the configuration of the Avro serializer that will be passed to the serializer by the producer. It simply points to where we store the schemas.
- ❸ `Customer` is our generated object. We tell the producer that our records will contain `Customer` as the value.
- ❹ `Customer` class is not a regular Java class (plain old Java object, or POJO) but rather a specialized Avro object, generated from a schema using Avro code generation. The Avro serializer can only serialize Avro objects, not POJO. Generating Avro classes can be done either using the *avro-tools.jar* or the Avro Maven plug-in, both part of Apache Avro. See the [Apache Avro Getting Started \(Java\) guide](#) for details on how to generate Avro classes.
- ❺ We also instantiate `ProducerRecord` with `Customer` as the value type, and pass a `Customer` object when creating the new record.
- ❻ That's it. We send the record with our `Customer` object, and `KafkaAvroSerializer` will handle the rest.

Avro also allows you to use generic Avro objects, that are used as key-value maps, rather than generated Avro objects with getters and setters that match the schema that was used to generate them. To use generic Avro objects, you just need to provide the schema:

```
Properties props = new Properties();
props.put("bootstrap.servers", "localhost:9092");
props.put("key.serializer",
    "io.confluent.kafka.serializers.KafkaAvroSerializer"); ❶
props.put("value.serializer",
    "io.confluent.kafka.serializers.KafkaAvroSerializer");
props.put("schema.registry.url", url); ❷

String schemaString =
    "{ \"namespace\": \"customerManagement.avro\",
      \"type\": \"record\", \" + ❸
      \"name\": \"Customer\", \" +
      \"fields\": [ \" +
        { \"name\": \"id\", \"type\": \"int\" }, \" +
        { \"name\": \"name\", \"type\": \"string\" }, \" +
        { \"name\": \"email\", \"type\": \" + \"[\"null\", \"string\"], \" +
        \"default\": \"null\" } \" +
      \" ] }";
Producer<String, GenericRecord> producer =
    new KafkaProducer<String, GenericRecord>(props); ❹

Schema.Parser parser = new Schema.Parser();
Schema schema = parser.parse(schemaString);

for (int nCustomers = 0; nCustomers < customers; nCustomers++) {
    String name = "exampleCustomer" + nCustomers;
    String email = "example " + nCustomers + "@example.com";

    GenericRecord customer = new GenericData.Record(schema); ❺
    customer.put("id", nCustomers);
    customer.put("name", name);
    customer.put("email", email);
```

```

        ProducerRecord<String, GenericRecord> data =
            new ProducerRecord<>("customerContacts", name, customer);
        producer.send(data);
    }

```

- ❶ We still use the same `KafkaAvroSerializer`.
- ❷ And we provide the URI of the same Schema Registry.
- ❸ But now we also need to provide the Avro schema, since it is not provided by an Avro-generated object.
- ❹ Our object type is an Avro `GenericRecord`, which we initialize with our schema and the data we want to write.
- ❺ Then the value of the `ProducerRecord` is simply a `GenericRecord` that contains our schema and data. The serializer will know how to get the schema from this record, store it in the Schema Registry, and serialize the object data.

Partitions

In previous examples, the `ProducerRecord` objects we created included a topic name, key, and value. Kafka messages are key-value pairs, and while it is possible to create a `ProducerRecord` with just a topic and a value, with the key set to `null` by default, most applications produce records with keys. Keys serve two goals: they are additional information that gets stored with the message, and they are typically also used to decide which one of the topic partitions the message will be written to (keys also play an important role in compacted topics—we'll discuss those in [Chapter 6](#)). All messages with the same key will go to the same partition. This means that if a process is reading only a subset of the partitions in a topic (more on that in [Chapter 4](#)), all the records for a single key will be read by the same process. To create a key-value record, you simply create a `ProducerRecord` as follows:

```

ProducerRecord<String, String> record =
    new ProducerRecord<>("CustomerCountry", "Laboratory Equipment", "USA");

```

When creating messages with a null key, you can simply leave the key out:

```

ProducerRecord<String, String> record =
    new ProducerRecord<>("CustomerCountry", "USA"); ❶

```

- ❶ Here, the key will simply be set to `null`.

When the key is `null` and the default partitioner is used, the record will be sent to one of the available partitions of the topic at random. A round-robin algorithm will be used to balance the messages among the partitions. Starting in the Apache Kafka 2.4 producer, the round-robin algorithm used in the default partitioner when handling null keys is sticky. This means that it will fill a batch of messages sent to a single partition before switching to the next partition. This allows sending the same number of messages to Kafka in fewer requests, leading to lower latency and reduced CPU utilization on the broker.

If a key exists and the default partitioner is used, Kafka will hash the key (using its own hash algorithm, so hash values will not change when Java is upgraded) and use the result to map the message to a specific partition. Since it is important that a key is always mapped to the same partition, we use all the partitions in the topic to calculate the mapping—not just the available partitions. This means that if a specific partition is unavailable when you write data to it, you might get an error. This is fairly rare, as you will see in [Chapter 7](#) when we discuss Kafka’s replication and availability.

In addition to the default partitioner, Apache Kafka clients also provide `RoundRobinPartitioner` and `UniformStickyPartitioner`. These provide random partition assignment and sticky random partition assignment even when messages have keys. These are useful when keys are important for the consuming application (for example, there are ETL applications that use the key from Kafka records as the primary key when loading data from Kafka to a relational database), but the workload may be skewed, so a single key may have a disproportionately large workload. Using the `UniformStickyPartitioner` will result in an even distribution of workload across all partitions.

When the default partitioner is used, the mapping of keys to partitions is consistent only as long as the number of partitions in a topic does not change. So as long as the number of partitions is constant, you can be sure that, for example, records regarding user 045189 will always get written to partition 34. This allows all kinds of optimization when reading data from partitions. However, the moment you add new partitions to the topic, this is no longer guaranteed—the old records will stay in partition 34 while new records may get written to a different partition. When partitioning keys is important, the easiest solution is to create topics with sufficient partitions (the Confluent blog contains suggestions on how to [choose the number of partitions](#)) and never add partitions.

Implementing a custom partitioning strategy

So far, we have discussed the traits of the default partitioner, which is the one most commonly used. However, Kafka does not limit you to just hash partitions, and sometimes there are good reasons to partition data differently. For example, suppose that you are a B2B vendor and your biggest customer is a company that manufactures handheld devices called Bananas. Suppose that you do so much business with customer “Banana” that over 10% of your daily transactions are with this customer. If you use default hash partitioning, the Banana records will get allocated to the same partition as other accounts, resulting in one partition being much larger than the rest. This can cause servers to run out of space, processing to slow down, etc. What we really want is to give Banana its own partition and then use hash partitioning to map the rest of the accounts to all other partitions.

Here is an example of a custom partitioner:

```
import org.apache.kafka.clients.producer.Partitioner;
import org.apache.kafka.common.Cluster;
import org.apache.kafka.common.PartitionInfo;
import org.apache.kafka.common.record.InvalidRecordException;
import org.apache.kafka.common.utils.Utils;

public class BananaPartitioner implements Partitioner {

    public void configure(Map<String, ?> configs) {} ❶
```

```

    public int partition(String topic, Object key, byte[] keyBytes,
                        Object value, byte[] valueBytes,
                        Cluster cluster) {
        List<PartitionInfo> partitions = cluster.partitionsForTopic(topic);
        int numPartitions = partitions.size();

        if ((keyBytes == null) || (!(key instanceof String))) ❷
            throw new InvalidRecordException("We expect all messages " +
                                             "to have customer name as key");

        if (((String) key).equals("Banana"))
            return numPartitions - 1; // Banana will always go to last partition

        // Other records will get hashed to the rest of the partitions
        return Math.abs(Utils.murmur2(keyBytes)) % (numPartitions - 1);
    }

    public void close() {}
}

```

- ❶ Partitioner interface includes `configure`, `partition`, and `close` methods. Here we only implement `partition`, although we really should have passed the special customer name through `configure` instead of hardcoding it in `partition`.
- ❷ We only expect `String` keys, so we throw an exception if that is not the case.

Headers

Records can, in addition to key and value, also include headers. Record headers give you the ability to add some metadata about the Kafka record, without adding any extra information to the key/value pair of the record itself. Headers are often used for lineage to indicate the source of the data in the record, and for routing or tracing messages based on header information without having to parse the message itself (perhaps the message is encrypted and the router doesn't have permissions to access the data).

Headers are implemented as an ordered collection of key/value pairs. The keys are always a `String`, and the values can be any serialized object—just like the message value.

Here is a small example that shows how to add headers to a `ProduceRecord`:

```

ProducerRecord<String, String> record =
    new ProducerRecord<>("CustomerCountry", "Precision Products", "France");

record.headers().add("privacy-level", "YOLO".getBytes(StandardCharsets.UTF_8));

```

Interceptors

There are times when you want to modify the behavior of your Kafka client application without modifying its code, perhaps because you want to add identical behavior to all applications in the organization. Or perhaps you don't have access to the original code.

Kafka's `ProducerInterceptor` interface includes two key methods:

```
ProducerRecord<K, V> onSend(ProducerRecord<K, V>
record)
```

This method will be called before the produced record is sent to Kafka, indeed before it is even serialized. When overriding this method, you can capture information about the sent record and even modify it. Just be sure to return a valid `ProducerRecord` from this method. The record that this method returns will be serialized and sent to Kafka.

```
void onAcknowledgement(RecordMetadata metadata,
Exception exception)
```

This method will be called if and when Kafka responds with an acknowledgment for a send. The method does not allow modifying the response from Kafka, but you can capture information about the response.

Common use cases for producer interceptors include capturing monitoring and tracing information; enhancing the message with standard headers, especially for lineage tracking purposes; and redacting sensitive information.

Here is an example of a very simple producer interceptor. This one simply counts the messages sent and acks received within specific time windows:

```
public class CountingProducerInterceptor implements ProducerInterceptor {

    ScheduledExecutorService executorService =
        Executors.newSingleThreadScheduledExecutor();
    static AtomicLong numSent = new AtomicLong(0);
    static AtomicLong numAked = new AtomicLong(0);

    public void configure(Map<String, ?> map) {
        Long windowSize = Long.valueOf(
            (String) map.get("counting.interceptor.window.size.ms")); ❶
        executorService.scheduleAtFixedRate(CountingProducerInterceptor::run,
            windowSize, windowSize, TimeUnit.MILLISECONDS);
    }

    public ProducerRecord onSend(ProducerRecord producerRecord) {
        numSent.incrementAndGet();
        return producerRecord; ❷
    }

    public void onAcknowledgement(RecordMetadata recordMetadata, Exception e) {
        numAked.incrementAndGet(); ❸
    }

    public void close() {
        executorService.shutdownNow(); ❹
    }

    public static void run() {
        System.out.println(numSent.getAndSet(0));
        System.out.println(numAked.getAndSet(0));
    }
}
```

❶ `ProducerInterceptor` is a `Configurable` interface. You can override the `configure` method and setup before any other method is called. This method receives the entire producer configu-

ration, and you can access any configuration parameter. In this case, we added a configuration of our own that we reference here.

- ❷ When a record is sent, we increment the record count and return the record without modifying it.
- ❸ When Kafka responds with an ack, we increment the acknowledgment count and don't need to return anything.
- ❹ This method is called when the producer closes, giving us a chance to clean up the interceptor state. In this case, we close the thread we created. If you opened file handles, connections to remote data stores, or similar, this is the place to close everything and avoid leaks.

As we mentioned earlier, producer interceptors can be applied without any changes to the client code. To use the preceding interceptor with `kafka-console-producer`, an example application that ships with Apache Kafka, follow these three simple steps:

1. Add your jar to the classpath:

```
export
CLASSPATH=$CLASSPATH:~/target/CountProducerInterceptor-
1.0-SNAPSHOT.jar
```

2. Create a config file that includes:

```
interceptor.classes=com.shapira.examples.interceptors.C
ountProducerInterceptor
counting.interceptor.window.size.ms=10000
```

3. Run the application as you normally would, but make sure to include the configuration that you created in the previous step:

```
bin/kafka-console-producer.sh --broker-list
localhost:9092 --topic interceptor-test --
producer.config producer.config
```

Quotas and Throttling

Kafka brokers have the ability to limit the rate at which messages are produced and consumed. This is done via the quota mechanism. Kafka has three quota types: produce, consume, and request. Produce and consume quotas limit the rate at which clients can send and receive data, measured in bytes per second. Request quotas limit the percentage of time the broker spends processing client requests.

Quotas can be applied to all clients by setting default quotas, specific client-ids, specific users, or both. User-specific quotas are only meaningful in clusters where security is configured and clients authenticate.

The default produce and consume quotas that are applied to all clients are part of the Kafka broker configuration file. For example, to limit each producer to send no more than 2 MBps on average, add the following configuration to the broker configuration file:

```
quota.producer.default=2M.
```

While not recommended, you can also configure specific quotas for certain clients that override the default quotas in the broker configuration file. To allow clientA to produce 4 MBps and clientB 10 MBps, you can use the following: `quota.`

```
producer.override="clientA:4M,clientB:10M"
```

Quotas that are specified in Kafka's configuration file are static, and you can only modify them by changing the configuration and then restarting all the brokers. Since new clients can arrive at any time, this is very inconvenient. Therefore the usual method of applying quotas to specific clients is through dynamic configuration that can be set using `kafka-config.sh` or the AdminClient API.

Let's look at few examples:

```
bin/kafka-configs --bootstrap-server localhost:9092 --alter --add-config 'producer_byte_rate=1024'
bin/kafka-configs --bootstrap-server localhost:9092 --alter --add-config 'producer_byte_rate=1024'
bin/kafka-configs --bootstrap-server localhost:9092 --alter --add-config 'consumer_byte_rate=2048'
```

- ❶ Limiting clientC (identified by client-id) to produce only 1024 bytes per second
- ❷ Limiting user1 (identified by authenticated principal) to produce only 1024 bytes per second and consume only 2048 bytes per second.
- ❸ Limiting all users to consume only 2048 bytes per second, except users with more specific override. This is the way to dynamically modify the default quota.

When a client reaches its quota, the broker will start throttling the client's requests to prevent it from exceeding the quota. This means that the broker will delay responses to client requests; in most clients this will automatically reduce the request rate (since the number of in-flight requests is limited) and bring the client traffic down to a level allowed by the quota. To protect the broker from misbehaved clients sending additional requests while being throttled, the broker will also mute the communication channel with the client for the period of time needed to achieve compliance with the quota.

The throttling behavior is exposed to clients via `produce-throttle-time-avg`, `produce-throttle-time-max`, `fetch-throttle-time-avg`, and `fetch-throttle-time-max`, the average and the maximum amount of time a produce request and fetch request was delayed due to throttling. Note that this time can represent throttling due to produce and consume throughput quotas, request time quotas, or both. Other types of client requests can only be throttled due to request time quotas, and those will also be exposed via similar metrics.

WARNING

If you use `async Producer.send()` and continue to send messages at a rate that is higher than the rate the broker can accept (whether due to quotas or just plain old capacity), the messages will first be queued in the client memory. If the rate of sending continues to be higher than the rate of accepting messages, the client will eventually run out of buffer space for storing the excess messages and will block the next `Producer.send()` call. If the timeout delay is insufficient to let the broker catch up to the producer and clear some space in the buffer, eventually `Producer.send()` will throw `TimeoutException`. Alternatively, some of the records that were already placed in batches will wait for longer than `delivery.timeout.ms` and expire, resulting in calling the `send()` callback with a `TimeoutException`. It is therefore important to plan and monitor to make sure that the broker capacity over time will match the rate at which producers are sending data.

Summary

We began this chapter with a simple example of a producer—just 10 lines of code that send events to Kafka. We added to the simple example by adding error handling and experimenting with synchronous and asynchronous producing. We then explored the most important producer configuration parameters and saw how they modify the behavior of the producers. We discussed serializers, which let us control the format of the events we write to Kafka. We looked in-depth at Avro, one of many ways to serialize events but one that is very commonly used with Kafka. We concluded the chapter with a discussion of partitioning in Kafka and an example of an advanced custom partitioning technique.

Now that we know how to write events to Kafka, in [Chapter 4](#) we'll learn all about consuming events from Kafka.

¹ Image contributed to the Apache Kafka project by Sumant Tambe under the ASLv2 license terms.